



EQUIPO NARANJA

ASIGNACIÓN ESCRITA IV

Uso de árboles y grafos en la solución de problemas.
Complejidad temporal de algoritmos.

DATOS DEL EQUIPO	
Equipo	Naranja - Equipo #15
Integrantes Y Matriculas	Hugo E. Montero Fulcar Matricula 100082929 Erasmus José Minaya Taveras Matricula 100076710 Albert Daniel Montero Ramírez Matricula 100078731 Luis Ángel Mora Taveras Matricula 100076432 Fernando Enrique Mesón Acosta Matricula 100027782
Asignatura	Estructura de Datos
Institución	Universidad Abierta para Adultos (UAPA)
Fecha	5 de Junio 2026

Santiago de los Caballeros, República Dominicana
Junio 2026

Introducción

El presente informe corresponde a la Asignación Escrita IV de la asignatura Estructuras de Datos, enfocada en el uso de árboles y grafos para la solución de problemas, así como en el análisis de la complejidad temporal de los algoritmos implementados. Los ejercicios desarrollados pertenecen al Equipo Naranja y fueron codificados en lenguaje C++ aplicando el Paradigma Orientado a Objetos (POO).

Se desarrollaron cuatro programas: dos basados en árboles binarios de búsqueda (ABB) — un árbol de números primos y un árbol de palabras para un buscador — y dos basados en grafos no dirigidos con recorrido en anchura (BFS) — una red de computadoras y una red de distribución de documentos entre oficinas. Cada programa incluye un selector de idioma (español/inglés) que permite mostrar la interfaz, los menús y las salidas en ambos idiomas desde un mismo archivo fuente.

Para cada función se determinó su complejidad algorítmica empleando la notación Big-O, considerando el número de nodos (n) en los árboles, y el número de vértices (V) y aristas (E) en los grafos.

Marco teórico

Árbol Binario de Búsqueda (ABB)

Un Árbol Binario de Búsqueda es una estructura jerárquica en la que cada nodo posee a lo sumo dos hijos, cumpliéndose que todos los valores del subárbol izquierdo son menores que el nodo, y todos los del subárbol derecho son mayores. Esta propiedad permite que las operaciones de inserción y búsqueda tengan un costo proporcional a la altura del árbol (h): $O(\log n)$ en promedio cuando el árbol está balanceado y $O(n)$ en el peor caso cuando degenera en una lista. El recorrido inorden de un ABB devuelve sus elementos en orden ascendente.

Grafos y recorrido BFS

Un grafo es un conjunto de vértices (nodos) conectados por aristas. En un grafo no dirigido, las aristas no tienen sentido: si A se conecta con B, también B se conecta con A. La representación utilizada es la lista de adyacencia, eficiente en memoria para grafos dispersos. El recorrido en anchura (BFS, Breadth-First Search) visita los nodos por niveles a partir de un origen, lo que lo hace idóneo para calcular la distancia mínima (número de saltos) entre dos nodos y para simular procesos de propagación o reparto por cercanía. Su complejidad es $O(V + E)$.

Ejercicio 1 — Árbol de Números Primos (ABB)

Descripción del problema

Se construye un Árbol Binario de Búsqueda con los primeros 15 números primos. Los primos se insertan en un orden que produce un árbol balanceado, de modo que la altura y el número de hojas sean representativos; no obstante, el recorrido inorden los devuelve siempre en orden ascendente. El programa muestra los primos en inorden, calcula el número de hojas y determina la altura del árbol.

Estructura de datos utilizada

Se emplea una clase Nodo (con valor entero y punteros a hijo izquierdo y derecho) y una clase ArbolPrimos que encapsula la raíz y todas las operaciones recursivas sobre el árbol.

Funciones implementadas

- insertar(v): inserta un primo respetando la propiedad del ABB.
- mostrarInorden(): recorre el árbol en inorden e imprime los primos ordenados.
- hojas(): cuenta los nodos hoja (sin hijos).
- altura(): determina la altura del árbol en número de niveles.

Análisis de complejidad algorítmica (Big-O)

Función / Operación	Complejidad	Justificación
insertar(v)	$O(h) \approx O(\log n)$	Desciende un nivel por comparación; peor caso $O(n)$.
mostrarInorden()	$O(n)$	Visita cada uno de los n nodos exactamente una vez.
hojas()	$O(n)$	Recorre todo el árbol para contar las hojas.
altura()	$O(n)$	Recorre todo el árbol calculando la profundidad máxima.
General	$O(n \log n)$	Construcción de n inserciones; consultas en $O(n)$.

Código fuente (C++ / POO)

```
/*=====
EQUIPO NARANJA - ASIGNACION IV / ORANGE TEAM - ASSIGNMENT IV
Ejercicio: Arbol de Numeros Primos (ABB) / Prime Number Tree (BST)
Lenguaje: C++ - Paradigma Orientado a Objetos (POO) / OOP
-----
ANALISIS DE COMPLEJIDAD ALGORITMICA / ALGORITHMIC TIME COMPLEXITY
- insertar(v)      : O(h)  -> promedio O(log n), peor caso O(n)
- mostrarInorden() : O(n)  -> visita cada nodo una vez
- hojas()          : O(n)  -> recorre todo el arbol
- altura()         : O(n)  -> recorre todo el arbol
```

```

    COMPLEJIDAD GENERAL : construir el arbol = O(n log n) promedio;
                        consultas (recorridos) = O(n)
=====*/
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

int IDIOMA = 1; // 1 = Espanol, 2 = English
string t(const string& es, const string& en){ return IDIOMA == 1 ? es : en; }

// Nodo del arbol / Tree node
class Nodo {
public:
    int valor;
    Nodo* izq;
    Nodo* der;
    Nodo(int v) : valor(v), izq(nullptr), der(nullptr) {}
};

// Arbol Binario de Busqueda de primos / Binary Search Tree of primes
class ArbolPrimos {
private:
    Nodo* raiz;

    // Insercion recursiva / Recursive insertion -- O(h)
    Nodo* insertar(Nodo* nodo, int v){
        if(nodo == nullptr) return new Nodo(v);
        if(v < nodo->valor) nodo->izq = insertar(nodo->izq, v);
        else if(v > nodo->valor) nodo->der = insertar(nodo->der, v);
        return nodo; // se ignoran duplicados / duplicates ignored
    }

    // Recorrido inorden (orden ascendente) / Inorder traversal -- O(n)
    void inorden(Nodo* nodo){
        if(nodo == nullptr) return;
        inorden(nodo->izq);
        cout << nodo->valor << " ";
        inorden(nodo->der);
    }

    // Conteo de hojas / Count leaf nodes -- O(n)
    int contarHojas(Nodo* nodo){
        if(nodo == nullptr) return 0;
        if(nodo->izq == nullptr && nodo->der == nullptr) return 1;
        return contarHojas(nodo->izq) + contarHojas(nodo->der);
    }

    // Altura del arbol (en numero de niveles) / Tree height -- O(n)
    int altura(Nodo* nodo){
        if(nodo == nullptr) return 0;
        return 1 + max(altura(nodo->izq), altura(nodo->der));
    }

public:
    ArbolPrimos() : raiz(nullptr) {}
    void insertar(int v){ raiz = insertar(raiz, v); }
    void mostrarInorden(){ inorden(raiz); cout << endl; }
    int hojas(){ return contarHojas(raiz); }
    int altura(){ return altura(raiz); }
};

int main(){
    cout << "Seleccione idioma / Select language:\n";
    cout << "1) Espanol\n2) English\n> ";
    cin >> IDIOMA;
    if(IDIOMA != 2) IDIOMA = 1;

    ArbolPrimos arbol;

```

```

/* Primeros 15 numeros primos. Se insertan en un orden que produce un
   arbol balanceado, de modo que la altura y el numero de hojas sean
   representativos. El inorden siempre los devuelve ordenados.
   First 15 primes inserted in a balanced order; inorder still returns
   them sorted. */
int primos[] = {19,7,37,3,13,29,43,2,5,11,17,23,31,41,47};
for(int p : primos) arbol.insertar(p);

cout << "\n=== " << t("ARBOL DE NUMEROS PRIMOS (ABB)",
                      "PRIME NUMBER TREE (BST)") << " ===\n";

cout << t("a) Primos en inorden: ", "a) Primes in inorder: ");
arbol.mostrarInorden();

cout << t("b) Numero de hojas: ", "b) Number of leaf nodes: ")
    << arbol.hojas() << endl;

cout << t("c) Altura del arbol (niveles): ", "c) Height of the tree (levels): ")
    << arbol.altura() << endl;

return 0;
}

```

Anexo: compilación y ejecución

Salida esperada: a) inorden → 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47; b) número de hojas → 8; c) altura → 4 niveles. Ejecutar el programa dos veces (opción 1 = Español, opción 2 = English) y capturar ambas salidas.

[Insertar aquí la captura de pantalla de la ejecución]

Figura — Ejecución de Ejercicio 1 — Árbol de Números Primos (ABB) en OnlineGDB

Ejercicio 2 — Árbol de Palabras para Buscador (ABB)

Descripción del problema

Se implementa un Árbol Binario de Búsqueda que almacena palabras clave. El sistema permite insertar palabras, sugerirlas en orden alfabético (recorrido inorden), buscar coincidencias exactas y mostrar cuántas palabras hay por nivel del árbol.

Estructura de datos utilizada

Clase NodoP (con una cadena 'palabra' y punteros a sus dos hijos) y clase ArbolPalabras que encapsula la raíz y las operaciones. La comparación entre cadenas determina la ubicación de cada palabra.

Funciones implementadas

- insertar(p): inserta una palabra respetando el orden lexicográfico.
- sugerir(): recorrido inorden que lista las palabras en orden alfabético.
- buscar(p): búsqueda binaria de una coincidencia exacta.
- palabrasPorNivel(): cuenta cuántas palabras hay en cada nivel del árbol.

Análisis de complejidad algorítmica (Big-O)

Función / Operación	Complejidad	Justificación
insertar(p)	$O(h) \approx O(\log n)$	Comparación de cadenas por nivel; peor caso $O(n)$.
sugerir() [inorden]	$O(n)$	Lista las n palabras visitando cada nodo una vez.
buscar(p)	$O(h) \approx O(\log n)$	Descarta la mitad del árbol en cada comparación.
palabrasPorNivel()	$O(n)$	Recorre el árbol acumulando el conteo por nivel.
General	$O(n \log n)$	Inserción/búsqueda $O(\log n)$; recorridos $O(n)$.

Código fuente (C++ / POO)

```
/*=====
EQUIPO NARANJA - ASIGNACION IV / ORANGE TEAM - ASSIGNMENT IV
Ejercicio: Arbol de Palabras para Buscador (ABB) / Word Tree for Search Engine
Lenguaje: C++ - Paradigma Orientado a Objetos (POO) / OOP
=====
ANALISIS DE COMPLEJIDAD ALGORITMICA / ALGORITHMIC TIME COMPLEXITY
- insertar(p)      : O(h)  -> promedio O(log n), peor caso O(n)
- sugerir() [inorden] : O(n)  -> lista alfabetica de todas las palabras
- buscar(p)       : O(h)  -> promedio O(log n), peor caso O(n)
- palabrasPorNivel() : O(n)  -> recorre todo el arbol
COMPLEJIDAD GENERAL : insercion/busqueda = O(log n) promedio;
                    recorridos = O(n)
=====*/
#include <iostream>
#include <string>
#include <algorithm>
```

```

using namespace std;

int IDIOMA = 1; // 1 = Espanol, 2 = English
string t(const string& es, const string& en){ return IDIOMA == 1 ? es : en; }

// Nodo con una palabra clave / Node holding a keyword
class NodoP {
public:
    string palabra;
    NodoP* izq;
    NodoP* der;
    NodoP(string p) : palabra(p), izq(nullptr), der(nullptr) {}
};

// ABB de palabras / Word BST
class ArbolPalabras {
private:
    NodoP* raiz;

    NodoP* insertar(NodoP* n, string p){ // O(h)
        if(n == nullptr) return new NodoP(p);
        if(p < n->palabra) n->izq = insertar(n->izq, p);
        else if(p > n->palabra) n->der = insertar(n->der, p);
        return n; // duplicados ignorados / duplicates ignored
    }

    void inorden(NodoP* n){ // O(n)
        if(n == nullptr) return;
        inorden(n->izq);
        cout << n->palabra << " ";
        inorden(n->der);
    }

    bool buscar(NodoP* n, const string& p){ // O(h)
        if(n == nullptr) return false;
        if(p == n->palabra) return true;
        if(p < n->palabra) return buscar(n->izq, p);
        return buscar(n->der, p);
    }

    int altura(NodoP* n){ // O(n)
        if(n == nullptr) return 0;
        return 1 + max(altura(n->izq), altura(n->der));
    }

    void contarNivel(NodoP* n, int nivel, int conteo[]){ // O(n)
        if(n == nullptr) return;
        conteo[nivel]++;
        contarNivel(n->izq, nivel + 1, conteo);
        contarNivel(n->der, nivel + 1, conteo);
    }

public:
    ArbolPalabras() : raiz(nullptr) {}
    void insertar(string p){ raiz = insertar(raiz, p); }
    void sugerir(){ inorden(raiz); cout << endl; }
    bool buscar(string p){ return buscar(raiz, p); }

    void palabrasPorNivel(){
        int h = altura(raiz);
        if(h == 0){ cout << t("arbol vacio", "(empty tree)") << endl; return; }
        int* conteo = new int[h]();
        contarNivel(raiz, 0, conteo);
        for(int i = 0; i < h; i++){
            cout << t("Nivel ", "Level ") << i << ": "
                 << conteo[i] << t(" palabra(s)", " word(s)") << endl;
        }
        delete[] conteo;
    }
};

```

```

int main(){
    cout << "Seleccione idioma / Select language:\n";
    cout << "1) Espanol\n2) English\n> ";
    cin >> IDIOMA;
    if(IDIOMA != 2) IDIOMA = 1;

    ArbolPalabras arbol;

    // Palabras clave iniciales / initial keywords
    string iniciales[] = {"computadora","algoritmo","red","datos",
                          "programa","arbol","grafo","busqueda"};
    for(string w : iniciales) arbol.insertar(w);

    int op;
    do{
        cout << "\n=== " << t("ARBOL DE PALABRAS (BUSCADOR)",
                                "WORD TREE (SEARCH ENGINE)") << " ===\n";
        cout << "1) " << t("Insertar palabra", "Insert word") << "\n";
        cout << "2) " << t("Sugerir palabras (orden alfabetico)",
                                "Suggest words (alphabetical)") << "\n";
        cout << "3) " << t("Buscar coincidencia exacta", "Search exact match") << "\n";
        cout << "4) " << t("Mostrar palabras por nivel", "Show words per level") << "\n";
        cout << "0) " << t("Salir", "Exit") << "\n> ";
        cin >> op;

        if(op == 1){
            cout << t("Palabra (sin espacios): ", "Word (no spaces): ");
            string w; cin >> w; arbol.insertar(w);
            cout << t("Palabra insertada.", "Word inserted.") << endl;
        }
        else if(op == 2){
            cout << t("Sugerencias: ", "Suggestions: ");
            arbol.sugerir();
        }
        else if(op == 3){
            cout << t("Palabra a buscar: ", "Word to search: ");
            string w; cin >> w;
            cout << (arbol.buscar(w) ? t("ENCONTRADA", "FOUND")
                                : t("NO encontrada", "NOT found")) << endl;
        }
        else if(op == 4){
            arbol.palabrasPorNivel();
        }
    } while(op != 0);

    return 0;
}

```

Anexo: compilación y ejecución

El programa carga palabras iniciales (computadora, algoritmo, red, datos, programa, arbol, grafo, busqueda). Las palabras se leen sin espacios mediante cin para evitar problemas de buffer en OnlineGDB. Probar las opciones del menú y capturar la pantalla en español e inglés.

[Insertar aquí la captura de pantalla de la ejecución]

Figura — Ejecución de Ejercicio 2 — Árbol de Palabras para Buscador (ABB) en OnlineGDB

Ejercicio 3 — Red de Computadoras (Grafo no dirigido)

Descripción del problema

Se modela una red de computadoras como un grafo no dirigido. El sistema permite establecer conexiones entre computadoras, simular la propagación de una actualización mediante un recorrido BFS, y calcular el número mínimo de saltos (distancia BFS) entre dos nodos.

Estructura de datos utilizada

Clase RedComputadoras que utiliza una lista de adyacencia implementada con un `map<int, vector<int>>`, donde cada clave es una computadora y su valor es la lista de computadoras conectadas a ella.

Funciones implementadas

- `conectar(a, b)`: agrega una conexión bidireccional entre dos computadoras.
- `propagar(inicio)`: recorrido BFS que muestra el orden de propagación de la actualización.
- `saltosMinimos(origen, destino)`: BFS que calcula la distancia mínima en número de saltos.

Análisis de complejidad algorítmica (Big-O)

Función / Operación	Complejidad	Justificación
<code>conectar(a, b)</code>	$O(1)$	Inserción al final de dos listas de adyacencia.
<code>propagar(inicio)</code>	$O(V + E)$	BFS visita cada vértice y cada arista una vez.
<code>saltosMinimos(o, d)</code>	$O(V + E)$	BFS recorre el grafo hasta hallar el destino.
General	$O(V + E)$	Costo dominado por los recorridos BFS.

Código fuente (C++ / POO)

```
/*=====
EQUIPO NARANJA - ASIGNACION IV / ORANGE TEAM - ASSIGNMENT IV
Ejercicio: Red de Computadoras (grafo no dirigido) / Computer Network
Lenguaje: C++ - Paradigma Orientado a Objetos (POO) / OOP
=====
ANALISIS DE COMPLEJIDAD ALGORITMICA / ALGORITHMIC TIME COMPLEXITY
V = numero de nodos (vertices), E = numero de aristas (conexiones)
- conectar(a,b)      : O(1)      -> insercion en lista de adyacencia
- propagar(inicio)   : O(V+E)    -> recorrido BFS completo
- saltosMinimos(o,d) : O(V+E)    -> BFS calcula distancia minima
COMPLEJIDAD GENERAL : O(V+E) por cada operacion de recorrido
=====*/
#include <iostream>
#include <vector>
#include <queue>
#include <map>
```

```

#include <string>
using namespace std;

int IDIOMA = 1; // 1 = Espanol, 2 = English
string t(const string& es, const string& en){ return IDIOMA == 1 ? es : en; }

// Grafo no dirigido con lista de adyacencia / Undirected graph (adjacency list)
class RedComputadoras {
private:
    map<int, vector<int>> ady;

public:
    // a) Conexion entre computadoras / Connection between computers -- O(1)
    void conectar(int a, int b){
        ady[a].push_back(b);
        ady[b].push_back(a); // no dirigido / undirected
    }

    // b) BFS de propagacion de actualizacion / update propagation -- O(V+E)
    void propagar(int inicio){
        map<int,bool> visit;
        queue<int> q;
        q.push(inicio); visit[inicio] = true;
        cout << t("Orden de propagacion: ", "Propagation order: ");
        while(!q.empty()){
            int u = q.front(); q.pop();
            cout << u << " ";
            for(int v : ady[u])
                if(!visit[v]){ visit[v] = true; q.push(v); }
        }
        cout << endl;
    }

    // c) Saltos minimos entre dos nodos / minimum hops (BFS distance) -- O(V+E)
    int saltosMinimos(int origen, int destino){
        map<int,int> dist;
        map<int,bool> visit;
        queue<int> q;
        q.push(origen); visit[origen] = true; dist[origen] = 0;
        while(!q.empty()){
            int u = q.front(); q.pop();
            if(u == destino) return dist[u];
            for(int v : ady[u])
                if(!visit[v]){ visit[v] = true; dist[v] = dist[u] + 1; q.push(v); }
        }
        return -1; // sin conexion / no path
    }
};

int main(){
    cout << "Seleccione idioma / Select language:\n";
    cout << "1) Espanol\n2) English\n> ";
    cin >> IDIOMA;
    if(IDIOMA != 2) IDIOMA = 1;

    RedComputadoras red;

    // Conexiones de ejemplo / sample connections
    red.conectar(0,1); red.conectar(0,2);
    red.conectar(1,3); red.conectar(2,3);
    red.conectar(3,4);

    int op;
    do{
        cout << "\n=== " << t("RED DE COMPUTADORAS", "COMPUTER NETWORK") << " ===\n";
        cout << "1) " << t("Conectar dos computadoras", "Connect two computers") << "\n";
        cout << "2) " << t("Propagar actualizacion (BFS)", "Propagate update (BFS)") << "\n";
        cout << "3) " << t("Saltos minimos entre dos nodos",
            "Minimum hops between two nodes") << "\n";
        cout << "0) " << t("Salir", "Exit") << "\n> ";
        cin >> op;
    }

```

```

    if(op == 1){
        int a, b;
        cout << t("Nodo A: ", "Node A: "); cin >> a;
        cout << t("Nodo B: ", "Node B: "); cin >> b;
        red.conectar(a, b);
        cout << t("Computadoras conectadas.", "Computers connected.") << endl;
    }
    else if(op == 2){
        int s; cout << t("Nodo de inicio: ", "Start node: "); cin >> s;
        red.propagar(s);
    }
    else if(op == 3){
        int a, b;
        cout << t("Origen: ", "Source: "); cin >> a;
        cout << t("Destino: ", "Target: "); cin >> b;
        int d = red.saltoMinimos(a, b);
        if(d < 0) cout << t("No hay conexión entre los nodos.",
                           "No connection between nodes.") << endl;
        else      cout << t("Saltos minimos: ", "Minimum hops: ") << d << endl;
    }
} while(op != 0);

return 0;
}

```

Anexo: compilación y ejecución

El programa carga una red de ejemplo (0-1, 0-2, 1-3, 2-3, 3-4). Probar la propagación desde un nodo y el cálculo de saltos mínimos entre dos nodos. Capturar la ejecución en ambos idiomas.

[Insertar aquí la captura de pantalla de la ejecución]

Figura — Ejecución de Ejercicio 3 — Red de Computadoras (Grafo no dirigido) en OnlineGDB

Ejercicio 4 — Distribución de Documentos en Red de Oficinas

Descripción del problema

Se representa la red interna de distribución de documentos de una empresa como un grafo no dirigido, donde cada oficina es un nodo y cada ruta directa es una arista. El grafo se construye a partir de las conexiones ingresadas por el usuario; luego se muestra la lista de adyacencia y se realiza un recorrido BFS desde la oficina central (nodo 0) para simular el orden de entrega.

Estructura de datos utilizada

Clase RedOficinas que utiliza una lista de adyacencia implementada con un vector<vector<int>> de tamaño fijo igual al número de oficinas.

Funciones implementadas

- agregarConexion(a, b): registra una ruta directa entre dos oficinas.
- mostrarLista(): imprime la lista de adyacencia completa.
- bfsDistribucion(0): recorrido BFS desde la oficina central que genera el orden de entrega.

Análisis de complejidad algorítmica (Big-O)

Función / Operación	Complejidad	Justificación
agregarConexion(a, b)	$O(1)$	Inserción en dos listas de adyacencia.
Construcción (entrada)	$O(E)$	Se leen y registran las E rutas ingresadas.
mostrarLista()	$O(V + E)$	Recorre todas las oficinas y sus conexiones.
bfsDistribucion(0)	$O(V + E)$	BFS visita cada oficina y ruta una sola vez.
General	$O(V + E)$	Costo dominado por la construcción y el BFS.

Código fuente (C++ / POO)

```
/*=====
EQUIPO NARANJA - ASIGNACION IV / ORANGE TEAM - ASSIGNMENT IV
Ejercicio: Distribucion de Documentos en Red de Oficinas
          Document Distribution in an Office Network
Lenguaje: C++ - Paradigma Orientado a Objetos (POO) / OOP
=====
ANALISIS DE COMPLEJIDAD ALGORITMICA / ALGORITHMIC TIME COMPLEXITY
V = numero de oficinas (nodos), E = numero de rutas (aristas)
- agregarConexion(a,b) : O(1)      -> insercion en lista de adyacencia
- construccion (input) : O(E)      -> se leen E rutas
- mostrarLista()       : O(V+E)    -> imprime todas las listas
- bfsDistribucion(0)   : O(V+E)    -> recorrido BFS de entrega
COMPLEJIDAD GENERAL    : O(V+E)
=====*/
```

```

#include <iostream>
#include <vector>
#include <queue>
#include <string>
using namespace std;

int IDIOMA = 1; // 1 = Espanol, 2 = English
string t(const string& es, const string& en){ return IDIOMA == 1 ? es : en; }

// Grafo no dirigido de oficinas / Undirected office graph
class RedOficinas {
private:
    int numOficinas;
    vector<vector<int>> ady; // lista de adyacencia / adjacency list

public:
    RedOficinas(int n) : numOficinas(n), ady(n) {}

    // a) Conexion directa entre oficinas / direct route -- O(1)
    void agregarConexion(int a, int b){
        ady[a].push_back(b);
        ady[b].push_back(a); // ruta no dirigida / undirected route
    }

    // Mostrar lista de adyacencia / display adjacency list -- O(V+E)
    void mostrarLista(){
        cout << t("\n--- Lista de adyacencia ---\n",
                  "\n--- Adjacency list ---\n");
        for(int i = 0; i < numOficinas; i++){
            cout << t("Oficina ", "Office ") << i << ": ";
            for(int v : ady[i]) cout << v << " ";
            cout << endl;
        }
    }

    // b) BFS desde la oficina central (nodo 0) / BFS from central office -- O(V+E)
    void bfsDistribucion(int inicio){
        vector<bool> visit(numOficinas, false);
        queue<int> q;
        q.push(inicio); visit[inicio] = true;
        cout << t("Orden de entrega (BFS desde oficina ",
                  "Delivery order (BFS from office ") << inicio << "): ";
        while(!q.empty()){
            int u = q.front(); q.pop();
            cout << u << " ";
            for(int v : ady[u])
                if(!visit[v]){ visit[v] = true; q.push(v); }
        }
        cout << endl;
    }
};

int main(){
    cout << "Seleccione idioma / Select language:\n";
    cout << "1) Espanol\n2) English\n> ";
    cin >> IDIOMA;
    if(IDIOMA != 2) IDIOMA = 1;

    int n, m;
    cout << t("Numero de oficinas: ", "Number of offices: ");
    cin >> n;
    RedOficinas red(n);

    cout << t("Numero de conexiones (rutas): ", "Number of connections (routes): ");
    cin >> m;

    cout << t("Ingrese cada ruta como dos numeros de oficina (0 a ",
              "Enter each route as two office numbers (0 to ")
        << n - 1 << "):\n";

```

```

for(int i = 0; i < m; i++){
    int a, b;
    cout << t("Ruta ", "Route ") << i + 1 << ": ";
    cin >> a >> b;
    if(a >= 0 && a < n && b >= 0 && b < n){
        red.agregarConexion(a, b);
    } else {
        cout << t("Oficina invalida, intente de nuevo.",
                "Invalid office, please try again.") << endl;
        i--; // repetir esta ruta / retry this route
    }
}

red.mostrarLista();
red.bfsDistribucion(0); // oficina central = nodo 0 / central office = node 0

return 0;
}

```

Anexo: compilación y ejecución

Datos de prueba sugeridos: 5 oficinas, 4 rutas → 0 1, 0 2, 1 3, 2 4. Resultado: lista de adyacencia por oficina y orden de entrega BFS desde la oficina 0 → 0 1 2 3 4. Capturar la ejecución en español e inglés.

[Insertar aquí la captura de pantalla de la ejecución]

Figura — Ejecución de Ejercicio 4 — Distribución de Documentos en Red de Oficinas en OnlineGDB

Resumen general de complejidad

La siguiente tabla consolida la complejidad temporal predominante de cada programa desarrollado por el equipo:

Función / Operación	Complejidad	Justificación
1. Árbol de Números Primos	$O(n \log n) / O(n)$	Construcción y recorridos del ABB.
2. Árbol de Palabras	$O(n \log n) / O(n)$	Inserción/búsqueda y recorridos del ABB.
3. Red de Computadoras	$O(V + E)$	Conexiones $O(1)$ y recorridos BFS.
4. Red de Oficinas	$O(V + E)$	Construcción del grafo y BFS de entrega.

Conclusión

El desarrollo de estos cuatro ejercicios permitió aplicar de forma práctica los árboles binarios de búsqueda y los grafos no dirigidos a problemas concretos: ordenamiento y conteo de elementos, sugerencia y búsqueda de palabras, propagación de actualizaciones en una red y simulación del reparto de documentos entre oficinas.

El análisis de complejidad evidenció que las operaciones sobre un ABB balanceado son eficientes ($O(\log n)$ en inserción y búsqueda), mientras que los recorridos completos —tanto en árboles como en grafos— escalan de forma lineal: $O(n)$ en árboles y $O(V + E)$ en grafos representados mediante listas de adyacencia. La implementación bajo el Paradigma Orientado a Objetos favoreció la encapsulación y la reutilización del código, y el selector de idioma permitió cumplir el requisito de presentar la interfaz en español e inglés.